

Redis 高可用集群云原生实验手册

Redis 高可用集群云原生实验手册
(Docker Desktop K8s 适配版)

目录

1. 实验概述

- 1.1 实验目标
- 1.2 实验架构
- 1.3 前置知识
- 1.4 实验时长

2. 环境准备

- 2.1 环境检查
- 2.2 Docker Desktop 资源配置
- 2.3 WSL2 环境说明
- 2.4 创建工作目录

3. K8s 基础操作演练

- 3.1 Namespace 管理
- 3.2 Pod 与 Service 操作
- 3.3 ConfigMap 与 Secret
- 3.4 检查清单

4. GitLab 代码仓库搭建

- 4.1 GitLab 部署
- 4.2 创建项目仓库
- 4.3 克隆项目到本地
- 4.4 项目目录结构
- 4.5 Redis Docker 镜像构建
 - docker/redis/Dockerfile
 - 关键知识点
 - 构建镜像
- 4.6 GitLab CI/CD
 - .gitlab-ci.yml
- 4.7 提交代码

5. Redis 集群上 K8s

- 5.1 架构设计

Redis Sentinel 高可用架构

关键设计点

对比项：传统 vs K8s

5.2 K8s 资源详解

k8s/base/namespace.yaml

k8s/base/configmap.yaml

k8s/base/secret.yaml

k8s/base/service.yaml

k8s/base/statefulset.yaml

k8s/base/sentinel.yaml

5.3 部署到 K8s

5.4 验证主从复制

5.5 验证 Sentinel

5.6 外部访问

5.7 检查清单

6. Argo CD GitOps 部署

6.1 GitOps 理念

6.2 安装 Argo CD

6.3 Argo CD CLI (可选)

6.4 Argo CD Application

argocd/redis-app.yaml

创建 Application

6.5 GitOps 验证

实验 1：修改配置触发同步

实验 2：自动修复 (Self-Healing)

实验 3：回滚

7. Prometheus 监控体系

7.1 监控架构

7.2 安装 Prometheus Stack

7.3 部署 Redis Exporter

redis-exporter.yaml 说明

7.4 配置告警规则

告警规则表

7.5 Grafana 仪表盘

8. 故障演练与 SOP

8.1 故障实验

实验 1：Master 宕机

实验 2：配置错误

实验 3：模拟网络分区

8.2 SOP 故障排查流程

Redis Pod 故障

主从复制故障

Sentinel 故障

8.3 传统运维 vs K8s 云原生（总结）

9. Docker Desktop K8s 常见问题

Q1: Pod 一直 Pending, PVC 未绑定

Q2: ImagePullBackOff

Q3: NodePort 访问不通

Q4: Argo CD 无法连接 GitLab

Q5: Pod 调度失败（资源不足）

Q6: Docker Desktop K8s 环境重置

10. 附录：课时安排

实验检查清单

Redis 高可用集群云原生实验手册

(Docker Desktop K8s 适配版)

目录

1. 实验概述
 2. 环境准备
 3. K8s 基础操作演练
 4. GitLab 代码仓库搭建
 5. Redis 集群上 K8s
 6. Argo CD GitOps 部署
 7. Prometheus 监控体系
 8. 故障演练与 SOP
 9. Docker Desktop K8s 常见问题
 10. 附录：课时安排
-

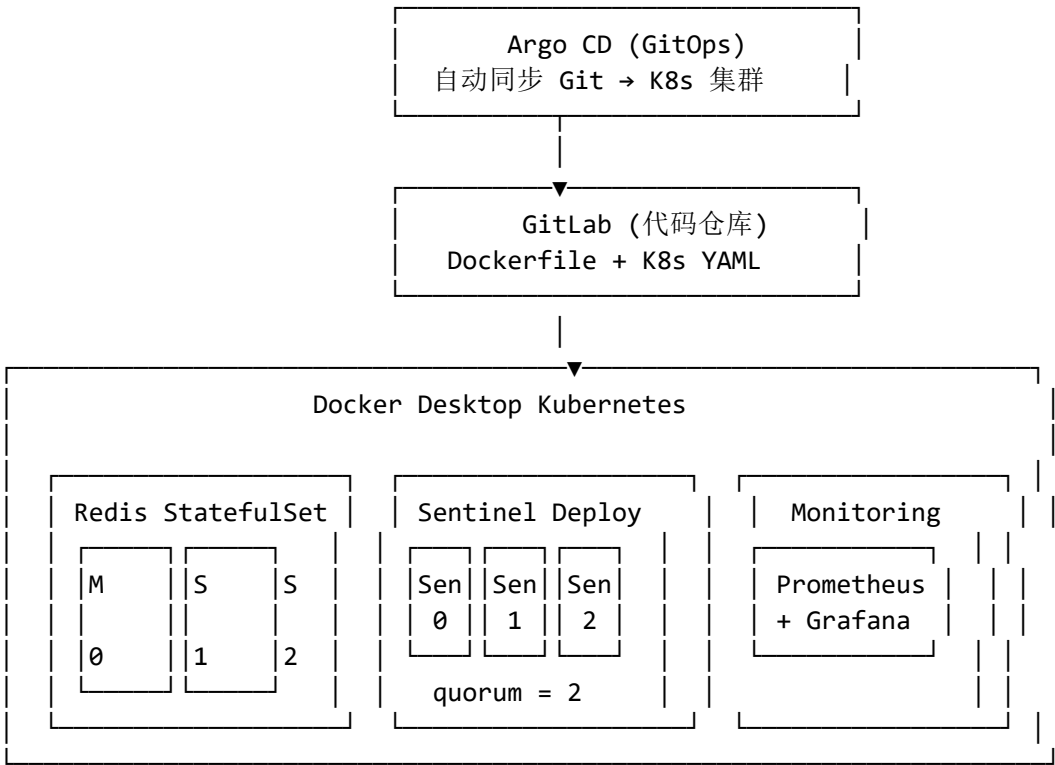
1. 实验概述

1.1 实验目标

传统运维方式	K8s 云原生方式
手动安装 Redis (yum/apt)	Docker 镜像 + StatefulSet 部署
Keepalived + VIP 实现高可用	Redis Sentinel 自动故障转移
redis.conf 手动管理	ConfigMap + Secret 配置中心化
手动脚本监控	Prometheus Exporter 自动采集
scp 分发 + 手动扩容	K8s replicas 一行扩缩
手动恢复	PVC 持久化 + StatefulSet 自愈

本实验将带你完成：**Redis 主从集群** → **Sentinel 哨兵高可用** → **K8s 部署** → **GitOps 持续交付** → **Prometheus 监控** 的完整链路。

1.2 实验架构



1.3 前置知识

- 了解 Docker 基础概念（镜像、容器）
- 了解 K8s 基础资源（Pod、Service、Namespace、ConfigMap、Secret）
- 了解 Redis 基本操作（redis-cli、主从复制概念）

- 了解 Git 基础命令

1.4 实验时长

总计约 **12~14 课时**（含实操时间）

2. 环境准备

2.1 环境检查

```
# 1. 验证 Docker
docker --version
docker info

# 2. 验证 K8s
kubectl get nodes
# 应显示 docker-desktop 状态为 Ready

# 3. 查看系统 Pod 状态
kubectl get pods -A
# kube-system 命名空间下的 Pod 应全部 Running

# 4. 检查 StorageClass
kubectl get storageclass
# Docker Desktop 默认提供 hostpath StorageClass
```

说明：Docker Desktop 内置的 Kubernetes 是一个单节点 K8s，master 和 worker 合一。StorageClass 使用 hostpath provisioner，PVC 会自动创建 PV。NodePort Service 可通过 localhost 直接访问。

2.2 Docker Desktop 资源配置

进入 Docker Desktop → Settings → Resources → Advanced：

资源项	推荐值
CPUs	4 核
Memory	8 GB
Swap	2 GB

资源项	推荐值
Disk image size	60 GB+

修改后点击 **Apply & Restart**。

2.3 WSL2 环境说明

- Docker Desktop 基于 WSL2，建议在 WSL2 Ubuntu 终端中操作
- 工作目录建议放在 WSL 内部（~/k8s-lab/），性能优于 /mnt/c/ 跨文件系统
- Windows PowerShell / CMD 中的 kubectl 与 WSL 共享同一配置

2.4 创建工作目录

```
mkdir -p ~/k8s-lab/redis-ha  
cd ~/k8s-lab/redis-ha
```

3. K8s 基础操作演练

3.1 Namespace 管理

```
# 创建实验用命名空间  
kubectl create namespace redis-ha  
kubectl create namespace gitlab  
kubectl create namespace argocd  
kubectl create namespace monitoring
```

```
# 查看所有命名空间  
kubectl get ns
```

3.2 Pod 与 Service 操作

```
# 1. 创建一个测试 Pod  
kubectl run nginx-test --image=nginx:alpine -n redis-ha
```

```
# 2. 查看 Pod 状态  
kubectl get pods -n redis-ha  
kubectl describe pod nginx-test -n redis-ha
```

```
# 3. 查看 Pod 日志
```

```
kubectl logs nginx-test -n redis-ha
```

4. 进入 Pod 内部

```
kubectl exec -it nginx-test -n redis-ha -- sh
```

5. 暴露 Service (NodePort)

```
kubectl expose pod nginx-test --port=80 --type=NodePort -n redis-ha
```

6. 查看 Service 和 NodePort

```
kubectl get svc nginx-test -n redis-ha
```

7. 通过 NodePort 访问 (替换 <NodePort> 为实际端口号)

```
# http://localhost:<NodePort>
```

```
curl localhost:<NodePort>
```

8. 清理测试资源

```
kubectl delete pod nginx-test -n redis-ha
```

```
kubectl delete svc nginx-test -n redis-ha
```

3.3 ConfigMap 与 Secret

1. 创建 ConfigMap

```
kubectl create configmap test-config \  
  --from-literal=app.name=myapp \  
  --from-literal=app.env=production \  
  -n redis-ha
```

2. 创建 Secret

```
kubectl create secret generic test-secret \  
  --from-literal=username=admin \  
  --from-literal=password=Admin@123 \  
  -n redis-ha
```

3. 查看资源

```
kubectl get configmap -n redis-ha  
kubectl get secret -n redis-ha  
kubectl describe configmap test-config  
kubectl describe secret test-secret -n redis-ha
```

4. 清理

```
kubectl delete configmap test-config -n redis-ha  
kubectl delete secret test-secret -n redis-ha
```

3.4 检查清单

- ☐ kubectl get nodes 显示 Ready
- ☐ 掌握 kubectl get / describe / logs / exec 命令
- ☐ 理解 Namespace 的作用

☐ 了解 NodePort Service 如何通过 localhost 访问

4. GitLab 代码仓库搭建

4.1 GitLab 部署

本实验使用 Docker 方式部署 GitLab CE（社区版），版本 16.10.0：

创建数据和日志目录

```
mkdir -p ~/k8s-lab/gitlab/{config,logs,data}
```

启动 GitLab 容器（首次启动需 5-10 分钟）

```
docker run -d \  
  --hostname gitlab.local \  
  --publish 8080:80 \  
  --publish 8443:443 \  
  --publish 2222:22 \  
  --name gitlab \  
  --restart always \  
  --volume ~/k8s-lab/gitlab/config:/etc/gitlab \  
  --volume ~/k8s-lab/gitlab/logs:/var/log/gitlab \  
  --volume ~/k8s-lab/gitlab/data:/var/opt/gitlab \  
  --shm-size 256m \  
  gitlab/gitlab-ce:16.10.0-ce.0
```

查看启动日志

```
docker logs -f gitlab
```

获取 root 初始密码

```
docker exec -it gitlab grep 'Password:' /etc/gitlab/initial_root_password
```

浏览器访问: <http://localhost:8080>

用户名: root

密码: 上面获取的密码

 **注意：**GitLab 需要至少 4GB 可用内存，否则启动非常慢甚至失败。

4.2 创建项目仓库

1. 登录 GitLab (<http://localhost:8080>)
2. 右上角 **Create a project** → **Create blank project**
3. Project name: redis-ha-k8s
4. 取消勾选 “Initialize repository with a README”

5. 点击 Create project

4.3 克隆项目到本地

```
cd ~/k8s-lab
```

```
# HTTP 方式克隆 (用你生成的 Access Token)
```

```
git clone http://root:<TOKEN>@localhost:8080/root/redis-ha-k8s.git
```

```
cd redis-ha-k8s
```

4.4 项目目录结构

```
redis-ha-k8s/
├── docker/
│   └── redis/
│       ├── Dockerfile          # Redis 7.2 镜像
│       └── redis.conf          # Redis 基础配置
├── k8s/
│   ├── base/
│   │   ├── namespace.yaml      # redis-ha 命名空间
│   │   ├── configmap.yaml      # Redis + Sentinel 配置
│   │   ├── secret.yaml         # Redis 密码
│   │   ├── service.yaml        # Headless + NodePort + Sentinel
│   │   ├── statefulset.yaml     # Redis 主从 StatefulSet
│   │   └── sentinel.yaml       # Sentinel 哨兵部署
│   └── monitoring/
│       ├── redis-exporter.yaml  # Prometheus Exporter
│       └── redis-alerts.yaml    # 告警规则
├── argocd/
│   └── redis-app.yaml          # ArgoCD Application
├── .gitlab-ci.yml              # CI/CD 流水线
└── README.md
```

4.5 Redis Docker 镜像构建

docker/redis/Dockerfile

```
FROM redis:7.2-alpine
```

```
# 复制 Redis 配置
```

```
COPY redis.conf /usr/local/etc/redis/redis.conf
```

```
# 健康检查
```

```
HEALTHCHECK --interval=10s --timeout=5s --retries=5 \
    CMD redis-cli ping || exit 1
```

EXPOSE 6379

关键知识点

为什么选 redis:7.2-alpine? - redis 官方镜像，版本 7.2 - alpine 变体：体积小 (~30MB)，安全，启动快 - 已内置 redis-server、redis-cli、redis-sentinel，无需额外安装

对比 MySQL Dockerfile - Redis 更简单：不需要 init.sql，不需要安装 Shell/Router - 主从复制通过启动参数 replicaof 一行搞定 - Sentinel 是 Redis 自带的，不需要额外组件

构建镜像

```
cd ~/k8s-lab/redis-ha-k8s
docker build -t redis-ha:7.2 ./docker/redis
```

验证镜像

```
docker images | grep redis-ha
```

🚩 **注意：**Docker Desktop 构建的镜像在 K8s 中可直接使用，但 YAML 中需设置 imagePullPolicy: IfNotPresent，防止去远程仓库拉取。

4.6 GitLab CI/CD

.gitlab-ci.yml

```
stages:
  - build
  - test

variables:
  IMAGE_NAME: "redis-ha/redis"
  IMAGE_TAG: "$CI_COMMIT_SHORT_SHA"

build-redis-image:
  stage: build
  image: docker:24.0
  services:
    - docker:24.0-dind
  script:
    - docker build -t $IMAGE_NAME:$IMAGE_TAG ./docker/redis
    - echo "Image built: $IMAGE_NAME:$IMAGE_TAG"
  only:
    - main
```

```
- develop
```

```
lint-k8s-manifests:
  stage: test
  image: alpine/helm:3.13.0
  script:
    - echo "K8s manifests lint check passed"
  only:
    - merge_requests
```

4.7 提交代码

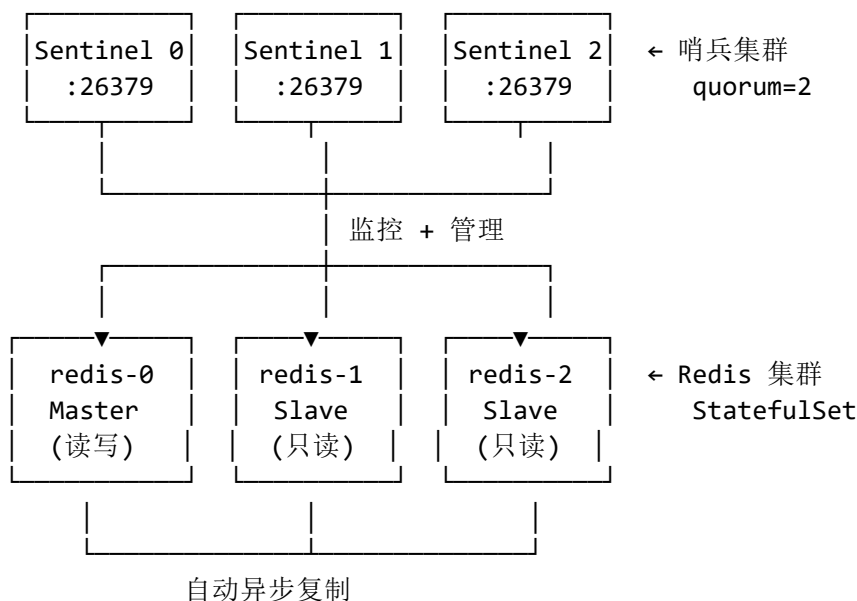
```
cd ~/k8s-lab/redis-ha-k8s
```

```
git add .
git commit -m "Initial commit: Redis HA K8s project"
git push origin main
```

5. Redis 集群上 K8s

5.1 架构设计

Redis Sentinel 高可用架构



关键设计点

设计要点	说明
StatefulSet redis-0 为初始 Master	InitContainer 根据 Pod 序号设置角色
redis-1, redis-2 自动 replicaof redis-0	启动时即建立主从关系
3 个 Sentinel 独立部署	避免与 Redis 实例同 Pod 相互影响
quorum=2	3 个 Sentinel 中至少 2 个同意才故障转移
Headless Service	提供稳定 DNS: redis-0.redis-headless.redis-ha.svc

对比项：传统 vs K8s

传统运维	K8s 云原生
手动安装 Redis: yum/apt	Docker 镜像一键部署, StatefulSet 管理
Keepalived + VIP 漂移	Redis Sentinel 自动故障发现和转移
redis.conf 散落在各节点	ConfigMap 集中管理, 一处修改处处生效
密码明文写在配置里	Secret 加密存储, 环境变量注入
通过 IP 写死主从关系	DNS 稳定标识, Pod 重启 IP 变了也能找到
手动脚本监控	Prometheus Exporter 自动采集指标
PVC + PV 实现数据持久化	数据不在 Pod 里, Pod 挂了数据还在

5.2 K8s 资源详解

k8s/base/namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: redis-ha
```

k8s/base/configmap.yaml

ConfigMap 存储两份配置：Redis 主从配置 + Sentinel 哨兵配置。

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: redis-config
  namespace: redis-ha
data:
  redis.conf: |
    bind 0.0.0.0
    port 6379
    protected-mode no
    save 900 1
    save 300 10
    save 60 10000
    dbfilename dump.rdb
    dir /data
    loglevel notice
    logfile ""
    maxmemory 256mb
    maxmemory-policy allkeys-lru

  sentinel.conf: |
    bind 0.0.0.0
    port 26379
    dir /tmp
    sentinel monitor mymaster redis-0.redis-headless.redis-
      ha.svc.cluster.local 6379 2
    sentinel down-after-milliseconds mymaster 5000
    sentinel failover-timeout mymaster 15000
    sentinel parallel-syncs mymaster 1
    sentinel auth-pass mymaster Redis@123456
```

参数解释： - sentinel monitor：监控 Master，quorum=2 表示至少 2 个 Sentinel 同意才判定 Master 下线 - down-after-milliseconds 5000：5 秒无响应即标记为主观下线 - failover-timeout 15000：故障转移最长等待 15 秒 - parallel-syncs 1：故障转移时，同时只有 1 个 Slave 同步新 Master

k8s/base/secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: redis-secret
  namespace: redis-ha
type: Opaque
stringData:
  redis-password: "Redis@123456"
```

🔴 **Secret vs ConfigMap：** 密码用 Secret 而非 ConfigMap。Secret 数据经过 base64 编码，且支持 etcd 加密存储。ConfigMap 用于明文配置。

k8s/base/service.yaml

三个 Service，各司其职：

```
---
# 1. Headless Service — StatefulSet 稳定 DNS
apiVersion: v1
kind: Service
metadata:
  name: redis-headless
  namespace: redis-ha
spec:
  ports:
    - port: 6379
      name: redis
  clusterIP: None           # Headless 的关键
  selector:
    app: redis

---
# 2. NodePort Service — 外部访问 Redis
apiVersion: v1
kind: Service
metadata:
  name: redis-service
  namespace: redis-ha
spec:
  ports:
    - port: 6379
      targetPort: 6379
      nodePort: 30379
      name: redis
  type: NodePort
  selector:
    app: redis

---
# 3. Sentinel Service — 哨兵间通信
apiVersion: v1
kind: Service
metadata:
  name: sentinel-service
  namespace: redis-ha
spec:
  ports:
    - port: 26379
      targetPort: 26379
      name: sentinel
  selector:
    app: sentinel
```

为什么需要 Headless Service? StatefulSet 中每个 Pod 需要稳定网络标识。Headless Service (clusterIP: None) 不为 Service 分配固定 IP，而是给每个 Pod 分配 DNS A 记录： - redis-0.redis-headless.redis-ha.svc.cluster.local - redis-1.redis-headless.redis-ha.svc.cluster.local - redis-2.redis-headless.redis-ha.svc.cluster.local

Sentinel 用 DNS 名监控 Master，而不是 IP，因为 Pod 重启 IP 会变。

k8s/base/statefulset.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
  namespace: redis-ha
spec:
  serviceName: redis-headless
  replicas: 3                # 1 Master + 2 Slave
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      # InitContainer: 根据 Pod 序号生成配置
      initContainers:
        - name: init-redis
          image: redis:7.2-alpine
          imagePullPolicy: IfNotPresent
          command:
            - sh
            - "-c"
            - |
              set -ex
              ordinal=${HOSTNAME##*-}

              # 复制基础配置
              cp /mnt/config/redis.conf /mnt/pod-config/redis.conf

              # 注入密码
              echo "requirepass ${REDIS_PASSWORD}" >> /mnt/pod-config/redis.conf
              echo "masterauth ${REDIS_PASSWORD}" >> /mnt/pod-config/redis.conf

              if [ "$ordinal" != "0" ]; then
                # 非 0 节点: 设为 Slave
                echo "replicaof redis-0.redis-headless.redis-ha.svc.cluster.local
                6379" \
```

```
    >> /mnt/pod-config/redis.conf
    echo "Configured as slave of redis-0"
else
    echo "Configured as master"
fi
env:
- name: REDIS_PASSWORD
  valueFrom:
    secretKeyRef:
      name: redis-secret
      key: redis-password
volumeMounts:
- name: pod-config
  mountPath: /mnt/pod-config
- name: redis-config
  mountPath: /mnt/config
```

主容器: Redis

```
containers:
- name: redis
  image: redis:7.2-alpine
  imagePullPolicy: IfNotPresent
  ports:
  - containerPort: 6379
    name: redis
  command:
  - redis-server
  - /mnt/pod-config/redis.conf
  volumeMounts:
  - name: redis-data
    mountPath: /data
  - name: pod-config
    mountPath: /mnt/pod-config
  livenessProbe:
    exec:
      command:
      - redis-cli
      - -a
      - "${REDIS_PASSWORD}"
      - ping
    initialDelaySeconds: 30
    periodSeconds: 10
  readinessProbe:
    exec:
      command:
      - redis-cli
      - -a
      - "${REDIS_PASSWORD}"
      - ping
    initialDelaySeconds: 10
    periodSeconds: 5
```



```

env:
- name: REDIS_PASSWORD
  valueFrom:
    secretKeyRef:
      name: redis-secret
      key: redis-password

volumes:
- name: redis-config
  configMap:
    name: redis-config
    items:
      - key: redis.conf
        path: redis.conf
- name: pod-config
  emptyDir: {}

volumeClaimTemplates:
- metadata:
    name: redis-data
  spec:
    accessModes: ["ReadWriteOnce"]
    resources:
      requests:
        storage: 5Gi

```

InitContainer 角色决定逻辑： 1. `${HOSTNAME##*-}` 提取 Pod 名末尾数字（redis-0 → 0, redis-1 → 1） 2. ordinal=0: Master（不做 replicaof） 3. ordinal≠0: Slave（replicaof redis-0）

这种方式避免手动创建 replication-init Job，Pod 启动即确定角色。

k8s/base/sentinel.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: sentinel
  namespace: redis-ha
spec:
  replicas: 3
  selector:
    matchLabels:
      app: sentinel
  template:
    metadata:
      labels:
        app: sentinel
    spec:
      affinity:

```

```
podAntiAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchLabels:
          app: sentinel
      topologyKey: kubernetes.io/hostname
initContainers:
- name: init-sentinel
  image: redis:7.2-alpine
  imagePullPolicy: IfNotPresent
  command:
    - sh
    - "-c"
    - |
      cp /mnt/config/sentinel.conf /mnt/sentinel-config/sentinel.conf
      echo "sentinel resolve-hostnames yes" >> /mnt/sentinel-
      config/sentinel.conf
      echo "sentinel announce-hostnames yes" >> /mnt/sentinel-
      config/sentinel.conf
  volumeMounts:
    - name: sentinel-config
      mountPath: /mnt/sentinel-config
    - name: sentinel-config-template
      mountPath: /mnt/config
containers:
- name: sentinel
  image: redis:7.2-alpine
  imagePullPolicy: IfNotPresent
  ports:
    - containerPort: 26379
      name: sentinel
  command:
    - redis-sentinel
    - /mnt/sentinel-config/sentinel.conf
  volumeMounts:
    - name: sentinel-config
      mountPath: /mnt/sentinel-config
  livenessProbe:
    exec:
      command:
        - redis-cli
        - -p
        - "26379"
        - ping
    initialDelaySeconds: 15
    periodSeconds: 10
volumes:
- name: sentinel-config-template
  configMap:
    name: redis-config
    items:
      - key: sentinel.conf
```

```

    path: sentinel.conf
- name: sentinel-config
  emptyDir: {}

```

Sentinel 关键设计： - 使用 Deployment 而非 StatefulSet（Sentinel 无状态，不需要稳定标识） - podAntiAffinity 将 Sentinel 分散调度，提高可用性 - quorum=2：3 个 Sentinel，至少 2 个同意才进行故障转移

5.3 部署到 K8s

```
cd ~/k8s-lab/redis-ha-k8s
```

1. 按顺序 *apply*

```

kubectl apply -f k8s/base/namespace.yaml
kubectl apply -f k8s/base/configmap.yaml
kubectl apply -f k8s/base/secret.yaml
kubectl apply -f k8s/base/service.yaml

```

2. 部署 *StatefulSet*

```
kubectl apply -f k8s/base/statefulset.yaml
```

3. 等待 Pod 就绪 (*redis-0* → *redis-1* → *redis-2* 依次启动)

```
kubectl get pods -n redis-ha -w
```

Ctrl+C 退出 *watch*

4. 部署 *Sentinel*

```
kubectl apply -f k8s/base/sentinel.yaml
```

5. 确认所有资源

```
kubectl get all -n redis-ha
```

✦ **启动顺序：** StatefulSet 按序号 0 → 1 → 2 顺序启动，确保 *redis-0* (Master) 先就绪，然后 *redis-1*、*redis-2* 作为 Slave 加入。

5.4 验证主从复制

1. 查看 *redis-0* 的角色

```
kubectl exec -it redis-0 -n redis-ha -- redis-cli -a Redis@123456 info replication | grep role
```

输出: *role:master*

2. 查看 *redis-1* 的角色

```
kubectl exec -it redis-1 -n redis-ha -- redis-cli -a Redis@123456 info replication | grep role
```

输出: *role:slave*

3. 在 Master 写入数据

```
kubectl exec -it redis-0 -n redis-ha -- redis-cli -a Redis@123456 set hello  
"World from K8s"
```

4. 在 Slave 读取数据

```
kubectl exec -it redis-1 -n redis-ha -- redis-cli -a Redis@123456 get hello  
# 输出: "World from K8s"
```

5. 在 Slave 尝试写入 (应被拒绝)

```
kubectl exec -it redis-1 -n redis-ha -- redis-cli -a Redis@123456 set test  
"fail"  
# 输出: (error) READONLY You can't write against a read only replica.
```

5.5 验证 Sentinel

1. 查看 Sentinel 监控的 Master

```
kubectl exec -it deploy/sentinel -n redis-ha -- redis-cli -p 26379 sentinel  
master mymaster
```

2. 查看所有 Slave

```
kubectl exec -it deploy/sentinel -n redis-ha -- redis-cli -p 26379 sentinel  
slaves mymaster
```

3. 查看所有 Sentinel

```
kubectl exec -it deploy/sentinel -n redis-ha -- redis-cli -p 26379 sentinel  
sentinels mymaster
```

5.6 外部访问

获取 NodePort

```
kubectl get svc redis-service -n redis-ha
```

使用 redis-cli 连接

```
redis-cli -h localhost -p 30379 -a Redis@123456
```

或使用 telnet

```
telnet localhost 30379
```

输入: PING

输出: +PONG

5.7 检查清单

- ☐ StatefulSet 3 个 Pod 全部 Running
- ☐ redis-0 为 Master, redis-1/redis-2 为 Slave
- ☐ Master 写入数据, Slave 能读取
- ☐ Sentinel 正确识别 Master 和 Slave
- ☐ localhost:30379 能连接 Redis

❑ PVC 已创建: `kubectl get pvc -n redis-ha`

6. Argo CD GitOps 部署

6.1 GitOps 理念

传统方式: 手动 `kubectl apply` → 不可追溯, 易出错

GitOps : Git 是唯一真相源 → Argo CD 自动同步到集群

工作流: 修改 Git YAML → `git push` → Argo CD 自动 `apply` → 集群更新

6.2 安装 Argo CD

1. 创建命名空间

```
kubectl create namespace argocd
```

2. 安装 Argo CD

```
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

3. 等待所有 Pod Running (约 2-3 分钟)

```
kubectl get pods -n argocd -w
```

4. 获取 admin 初始密码

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d  
echo ""
```

5. 暴露 Argo CD Server 为 NodePort

```
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "NodePort"}}'
```

6. 获取 NodePort

```
kubectl get svc argocd-server -n argocd
```

7. 浏览器访问

`https://localhost:<NodePort>`

用户名: admin

密码: 第 4 步的输出

⚠ Docker Desktop 中 Argo CD 用 HTTPS (自签证书), 浏览器提示不安全时点击 “高级” → “继续访问”。

6.3 Argo CD CLI（可选）

安装 CLI

```
curl -sSL -o argocd https://github.com/argoproj/argo-  
cd/releases/latest/download/argocd-linux-amd64  
sudo install -m 555 argocd /usr/local/bin/argocd
```

验证

```
argocd version --client
```

6.4 Argo CD Application

argocd/redis-app.yaml

```
apiVersion: argoproj.io/v1alpha1  
kind: Application  
metadata:  
  name: redis-ha  
  namespace: argocd  
spec:  
  project: default  
  source:  
    repoURL: 'http://gitlab.local/root/redis-ha-k8s.git'  
    targetRevision: main  
    path: k8s/base  
  destination:  
    server: 'https://kubernetes.default.svc'  
    namespace: redis-ha  
  syncPolicy:  
    automated:  
      prune: true          # Git 中删除的文件，集群中也删除  
      selfHeal: true       # 手动改集群的，自动回滚到 Git 状态  
    syncOptions:  
      - CreateNamespace=true # 自动创建 namespace
```

🔴 **repoURL 说明：** - GitLab 和 K8s 在同一台机器：用 Docker 内部地址或 host.docker.internal:8080 - 测试验证：kubect1 exec -it -n argocd <repo-server-pod> -- curl http://host.docker.internal:8080

创建 Application

```
kubect1 apply -f argocd/redis-app.yaml
```

查看状态

```
kubect1 get application -n argocd
```

应显示 *Healthy + Synced*

6.5 GitOps 验证

实验 1：修改配置触发同步

```
# 1. 修改 ConfigMap: maxmemory 从 256mb 改为 512mb
# 编辑 k8s/base/configmap.yaml

# 2. 提交并推送
git add k8s/base/configmap.yaml
git commit -m "Increase maxmemory to 512mb"
git push origin main

# 3. 观察 Argo CD 自动同步 (约 3 分钟内)
kubectl get application -n argocd -w

# 4. 验证 ConfigMap 已更新
kubectl get configmap redis-config -n redis-ha -o yaml | grep maxmemory

# 5. 重启 StatefulSet 使 Redis 加载新配置
kubectl rollout restart statefulset redis -n redis-ha
```

实验 2：自动修复 (Self-Healing)

```
# 1. 手动删除一个 Service (模拟误操作)
kubectl delete svc redis-service -n redis-ha

# 2. 观察 Argo CD 自动恢复 (1-2 分钟内)
kubectl get svc redis-service -n redis-ha
# Service 被自动重建
```

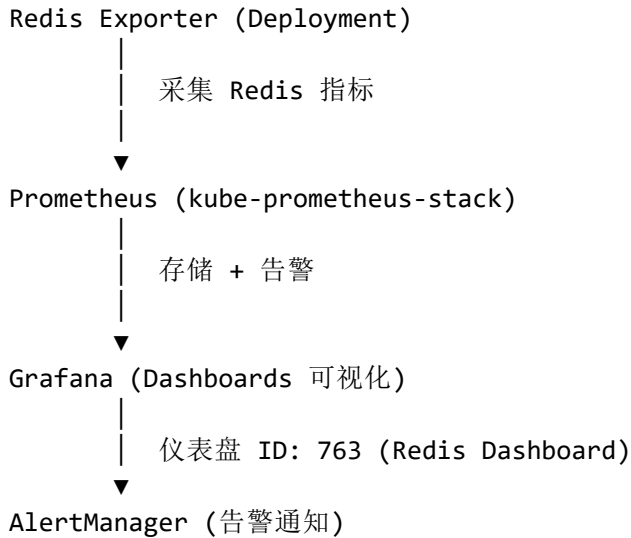
实验 3：回滚

```
# 方法一: Argo CD UI → redis-ha → HISTORY AND ROLLBACK → 选择版本 → Rollback

# 方法二: Git revert
git revert HEAD
git push origin main
```

7. Prometheus 监控体系

7.1 监控架构



7.2 安装 Prometheus Stack

1. 安装 Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
  bash
helm version
```

2. 添加 Helm 仓库

```
helm repo add prometheus-community https://prometheus-
  community.github.io/helm-charts
helm repo update
```

3. 安装 kube-prometheus-stack

```
helm install kube-prom-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false \
  --set grafana.service.type=NodePort \
  --set prometheus.service.type=NodePort
```

4. 等待 Pod 就绪 (约 2-3 分钟)

```
kubectl get pods -n monitoring -w
```

5. 获取 Grafana NodePort

```
kubectl get svc kube-prom-stack-grafana -n monitoring
```

6. 访问 Grafana

```
# http://localhost:<Grafana NodePort>
# 用户名: admin
# 密码: prom-operator
```


7.3 部署 Redis Exporter

1. 在 `redis-0` 上创建 `exporter` 用户 (可选)

```
kubectl exec -it redis-0 -n redis-ha -- redis-cli -a Redis@123456 ACL SETUSER
    exporter ON >Exporter@123456 +INFO +PING
```

2. 部署 `Exporter`

```
kubectl apply -f k8s/monitoring/redis-exporter.yaml
```

3. 验证

```
kubectl get pods -n monitoring -l app=redis-exporter
kubectl get servicemonitor -n monitoring
```

4. 检查指标

```
kubectl exec -it deploy/redis-exporter -n monitoring -- curl
    localhost:9121/metrics | head -20
```

redis-exporter.yaml 说明

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-exporter
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis-exporter
  template:
    spec:
      containers:
        - name: redis-exporter
          image: oliver006/redis_exporter:v1.59.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 9121
              name: metrics
          env:
            - name: REDIS_ADDR
              value: "redis://:Redis@123456@redis-service.redis-
                ha.svc.cluster.local:6379"
      ---
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: redis-exporter
  namespace: monitoring
  labels:
    release: kube-prom-stack # 匹配 Prometheus 配置
```

```
spec:
  selector:
    matchLabels:
      app: redis-exporter
  endpoints:
    - port: metrics
      interval: 15s
```

核心机制：ServiceMonitor 通过 release: kube-prom-stack 标签被 Prometheus 自动发现，无需手动配置 prometheus.yml。

7.4 配置告警规则

```
kubectl apply -f k8s/monitoring/redis-alerts.yaml
```

```
# 验证
kubectl get prometheusrule -n monitoring
```

告警规则表

告警名称	触发条件	严重级别	说明
RedisDown	redis_up == 0 持续 1 分钟	critical	Redis 实例宕机
RedisMasterDown	无 Master 节点超过 30 秒	critical	Sentinel 可能正在故障转移
RedisTooManyConnections	连接数 > maxclients 的 80%	warning	连接数过高
RedisRejectedConnections	有被拒绝的连接	warning	可能达到连接上限
RedisMemoryHigh	内存使用率 > 80%	warning	内存使用过高

7.5 Grafana 仪表盘

1. 登录 Grafana → Dashboards → Import
2. 输入 Dashboard ID: **763** (Redis Dashboard for Prometheus)
3. 选择 Prometheus 数据源 → Import
4. 查看 Redis 指标:

- QPS（每秒查询数）
 - 连接数
 - 内存使用率
 - 命中率
 - 主从复制延迟
-

8. 故障演练与 SOP

8.1 故障实验

实验 1: Master 宕机

1. 确认当前 Master 和数据

```
kubectl exec -it redis-0 -n redis-ha -- redis-cli -a Redis@123456 info  
replication | grep role  
kubectl exec -it redis-0 -n redis-ha -- redis-cli -a Redis@123456 keys '*'
```

2. 删除 redis-0（模拟宕机）

```
kubectl delete pod redis-0 -n redis-ha
```

3. 观察 StatefulSet 自动重建 Pod

```
kubectl get pods -n redis-ha -w
```

4. 观察 Sentinel 的故障转移

```
kubectl exec -it deploy/sentinel -n redis-ha -- redis-cli -p 26379 sentinel  
master mymaster
```

注意 Master 已切换到 redis-1 或 redis-2

5. 检查数据完整性

```
kubectl exec -it <new-master> -n redis-ha -- redis-cli -a Redis@123456 keys  
'*'
```

数据应完整存在

6. 查看 redis-0 恢复后的角色（变为 Slave）

```
kubectl exec -it redis-0 -n redis-ha -- redis-cli -a Redis@123456 info  
replication | grep role
```

关键观察： 1. Sentinel 在 5 秒内检测到 Master 宕机 2. 故障转移在约 15 秒内完成 3. 新 Master 的数据完整 4. 原 Master 恢复后自动成为 Slave，不会造成脑裂 5. PVC 保证了数据持久化，Pod 重启数据不丢

实验 2：配置错误

```
# 1. 修改 ConfigMap 中 redis.conf 的一个正确参数（如 maxmemory-policy）为错误值
kubectl edit configmap redis-config -n redis-ha

# 2. 重启 StatefulSet
kubectl rollout restart statefulset redis -n redis-ha

# 3. 观察 Pod 状态
kubectl get pods -n redis-ha
# Pod 可能 CrashLoopBackOff

# 4. 查看日志定位问题
kubectl logs redis-0 -n redis-ha
kubectl logs redis-0 -n redis-ha --previous

# 5. 恢复配置
# 修改 ConfigMap 回正确值 → restart
```

实验 3：模拟网络分区

```
# 1. 删除一个 Sentinel Pod
kubectl delete pod sentinel-xxx -n redis-ha

# 2. 观察 Deployment 自动恢复
kubectl get pods -n redis-ha -w

# 3. 检查 Sentinel 集群信息
kubectl exec -it deploy/sentinel -n redis-ha -- redis-cli -p 26379 sentinel
ckquorum mymaster
# ckquorum 检查是否达到 quorum
```

8.2 SOP 故障排查流程

Redis Pod 故障

步骤	命令	说明
1. 查看 Pod 状态	kubectl get pods -n redis-ha	确认哪个 Pod 异常
2. 查看日志	kubectl logs redis-0 -n redis-ha	最近日志
3. 查看上一次日志	kubectl logs redis-0 -n redis-ha --previous	Pod 重启前的日志

步骤	命令	说明
4. 查看 Events	kubectl describe pod redis-0 -n redis-ha	调度、拉镜像等问题
5. 检查 PVC	kubectl get pvc -n redis-ha	存储是否正常
6. 进入容器排查	kubectl exec -it redis- 0 -n redis-ha -- sh	直接查看容器内部
7. 检查 Service	kubectl get svc -n redis-ha	网络是否正常
8. DNS 解析测试	kubectl run -it dns- test -- image=busybox:1.28 --rm -- nslookup redis- service.redis- ha.svc.cluster.local	DNS 是否正常

主从复制故障

步骤	命令	说明
1. 查看复制状态	redis-cli -a <pw> info replication	确认角色和状态
2. 检查连接	redis-cli -a <pw> -h redis-1.redis-headless -p 6379 ping	节点间连通性
3. 检查密钥	kubectl get secret redis-secret -n redis- ha -o yaml	密码是否正确
4. 手动重连	redis-cli -a <pw> replicaof redis- 0.redis-headless 6379	手动建立主从

Sentinel 故障

步骤	命令	说明
1. 检查 Sentinel	redis-cli -p 26379 sentinel master mymaster	当前 Master 信息

步骤	命令	说明
2. 检查 quorum	redis-cli -p 26379 sentinel ckquorum mymaster	quorum 是否满足
3. 重置 Sentinel	redis-cli -p 26379 sentinel reset mymaster	重置监控（慎用）

8.3 传统运维 vs K8s 云原生（总结）

场景	传统运维	K8s 云原生
部署	每台机器 yum install + 手动配置	kubectl apply -f 一键部署
配置管理	每台机器单独修改 redis.conf	ConfigMap 一处修改，ConfigMap 更新后 Pod 需重启才生效
高可用	Keepalived + VIP 漂移	Sentinel 自动故障转移
监控	手动脚本 + 告警不易扩展	Prometheus Exporter + Grafana 开箱即用
扩容	手动安装 + 配置 + 加监控	修改 replicas 即可
故障恢复	手动登录服务器排查	Pod 自动重建，PVC 数据持久
数据安全	手动备份	PVC 快照 + Longhorn 备份（升级选项）

9. Docker Desktop K8s 常见问题

Q1: Pod 一直 Pending，PVC 未绑定

```
# 检查 StorageClass
kubectl get storageclass

# 如果没有默认 SC，Docker Desktop 需 Reset Kubernetes Cluster
# Docker Desktop → Troubleshoot → Reset Kubernetes Cluster
```

Q2: ImagePullBackOff

查看详细原因

```
kubectl describe pod <pod-name> -n redis-ha
```

解决方法:

1. 检查镜像是否已构建: `docker images | grep redis`

2. 确保 `imagePullPolicy` 设为 `IfNotPresent` (本地镜像不用拉取)

3. 检查镜像名和 `tag` 是否匹配

Q3: NodePort 访问不通

1. 确认 `Service` 的 `NodePort`

```
kubectl get svc redis-service -n redis-ha
```

2. 确认 `NodePort` 在 30000-32767 范围内

3. 确认 `Pod` 已 `Running`

4. `Docker Desktop` 有时 `NodePort` 不稳定

解决方法: `Docker Desktop` → `Troubleshoot` → `Reset Kubernetes Cluster`

或改用 `port-forward`:

```
kubectl port-forward svc/redis-service 6379:6379 -n redis-ha
```

Q4: Argo CD 无法连接 GitLab

1. 测试 `Argo CD repo-server` 到 `GitLab` 的连通性

```
kubectl exec -it -n argocd deploy/argocd-repo-server -- curl  
http://host.docker.internal:8080
```

2. 如果 `GitLab` 和 `K8s` 都在 `Docker Desktop` 内

`repoURL` 使用: `http://host.docker.internal:8080/root/redis-ha-k8s.git`

`host.docker.internal` 是 `Docker` 提供的访问宿主机的特殊 `DNS`

Q5: Pod 调度失败 (资源不足)

查看节点资源

```
kubectl describe nodes
```

解决方法:

1. `Docker Desktop` → `Settings` → `Resources` → 增加 `CPU/内存`

2. 删除不用的资源释放空间

3. 清理已停止的容器和镜像: `docker system prune -a`

Q6: Docker Desktop K8s 环境重置

如果 `K8s` 集群出问题, 重置方法:

1. `Docker Desktop` → `Settings` → `Kubernetes` → `Reset Kubernetes Cluster`

- # 2. 等待重置完成
- # 3. 保持以下资源不丢：
 - # - 代码和 `YAML` 文件（在 `Git` 里）
 - # - `Docker` 镜像（本地构建的需重建）
 - # - `PV` 数据（`reset` 后可能丢失，注意备份）

10. 附录：课时安排

阶段	内容	课时
环境准备	Docker Desktop 安装配置、K8s 启用	0.5
K8s 基础	Pod、Service、ConfigMap、Secret 实操	1
GitLab + CI	GitLab 部署、仓库创建、CI 流水线	2
Redis 上 K8s	StatefulSet、Sentinel、主从复制	3
Argo CD GitOps	Argo CD 安装、Application、自动同步实验	2
Prometheus 监控	kube-prometheus-stack、Exporter、Grafana 仪表盘	2
故障演练	Master 宕机、配置错误、SOP 流程	2
总结答疑	回顾架构、传统 vs 云原生对比	1.5
合计		14

实验检查清单

在实验结束前，确认以下各项：

- ☐ 能独立部署 Redis StatefulSet + Sentinel
- ☐ 理解 Headless Service 和 DNS 稳定标识的作用

- ☐ 理解 Sentinel 的 quorum 和故障转移机制
 - ☐ 能在 Master 写入、Slave 读取，验证主从复制
 - ☐ 能通过 Argo CD 自动部署和同步
 - ☐ 能通过 Grafana 查看 Redis 监控指标
 - ☐ 能独立完成 Master 宕机故障转移实验
 - ☐ 掌握 kubectl 常用命令和故障排查流程
-

v1.0 — Redis 高可用集群云原生实验手册 基于 Windows + Docker
Desktop Kubernetes 2026 年 7 月

本手册与传统运维的对应关系：

- Keepalived + VIP → Redis Sentinel 自动故障转移
- 手动安装 Redis → Docker 镜像 + K8s StatefulSet
- 散落配置文件 → ConfigMap 集中管理
- 明文密码 → Secret 加密存储
- 手动脚本监控 → Prometheus + Grafana
- scp 分发 + 手动部署 → Git + Argo CD GitOps

通过本实验，你将理解 K8s 如何解决传统运维中的痛点，以及 Cloud Native 架构的核心价值。